Computing (2024) 106:2389–2425
https://doi.org/10.1007/s00607-024-01292-z

REGULAR PAPER

Exploring the impact of chaos engineering with various user loads on cloud native applications: an exploratory empirical study

Amro Al-Said Ahmad[1] · **Lamis F. Al-Qora'n**[2] · **Ahmad Zayed**[2]

Received: 12 September 2023 / Accepted: 22 April 2024 / Published online: 5 May 2024
© The Author(s) 2024, corrected publication 2024

Abstract

One of the most popular models that provide computer resources today is cloud computing. Today's dynamic and successful platforms are created to take advantage of various resources available from service providers. Ensuring the performance and availability of such resources and services is a crucial problem. Any software system may be subject to faults that might propagate to cause failures. Such faults with the potential of contributing to failures are critical because they impair performance and result in a delayed reaction, which is regarded as a dependability problem. To ensure that critical faults can be discovered as soon as possible, the impact of such faults on the system must be tested. The performance and dependability of cloud-native systems are examined in this empirical study using fault injection, one of the chaos engineering techniques. The study explores the impacts and results of injecting various delay times into two cloud-native applications with diverse user numbers. The performance of the applications with various numbers of users is measured in relation to these delays, which accordingly reflects measuring the dependability of those systems. Firstly, the systems' architecture were identified, and serverless with two Lambda functions and containerised microservices applications were chosen, which depend on utilising and incorporating cloud-native services. Secondly, faults are injected in order to quantify performance attributes such as throughput and latency. The results of several controlled experiments carried out in real-world cloud environments provide exploratory empirical data, which promoted comparisons and statistical analysis that we utilised to identify the behaviour of the application while experiencing stress. Typical results from this investigation include an overall reduction in performance that is embodied in an increase in latency with injecting delays. However, a remarkable result is noticed at a particular delay in which defects and availability problems appear out of nowhere. These findings assist in highlighting the value of using chaos engineering in general and fault injection in particular to assess the dependability of cloud-native applications and to find unpredicted failures that could arise quickly from defects that aren't supposed to spread and result in dependability issues.

Extended author information available on the last page of the article

Keywords Chaos engineering · Fault injection · Dependability · Performance · Availability · Cloud-native systems

1 Introduction

Cloud-based apps are becoming increasingly common in various industries, such as healthcare [1], education [2],and many more [3, 4]. Each of these industries has different standards for quality; for instance, the healthcare industry prioritises availability and safety, whereas the education industry focuses mostly on usability and availability. On the other hand, all of these industries prioritise dependability and performance. Cloud computing (CC) facilitates the use of its services as an infrastructure for hosting and deploying software systems. This creates distributed systems with complicated and varied behaviours for software-based services. Additionally, CC enables businesses to develop modern distributed systems with high performance, scalability, pay-as-you-go, ubiquitously availability, and other cloud-based enabling features. This enables businesses to employ cloud-based services as an infrastructure for everyday systems, including embedded and critical systems like banking, telecommunication, transportation, and healthcare [1–4]. However, the increasing dependence on cloud services has resulted in a number of problems with service availability, performance, and reliability [5, 6]. The dependability of the cloud service itself affects the quality of cloud-based services as well; problems with cloud service dependability are likely to have a negative impact on service quality and delivery.

As complex systems become increasingly common and more companies rely on cloud solutions to host their services, evaluating these systems' dependability, resilience, and reliability is crucial. In such a real-time system, the dependability of cloud computing solutions refers to the capability of providing reliable services within a specified time frame [7] by exposing the system to potential attacks and controlled failures. This has the potential to help in developing applications that respond to the growing or changing workload demands and are resilient enough to withstand service and fault disruptions. Many large technology companies have developed tools and conducted fault-mode testing to ensure their products can withstand faults. One example is Chaos engineering, which is based on Netflix's Chaos Monkey tool and involves conducting runtime experiments to assess a system's resilience [8]. Other companies, including Facebook, Amazon, Microsoft, and Google, have adopted similar approaches to evaluate the resilience of their products.

Cloud resources such as hardware and software tend to fail and are prone to failures at any time, which can affect the functionality of the other resources [9]. Therefore, cloud environments experience numerous failures [10], and the key to preventing such failures is the systematic integration of dependability within the development process. Cloud-native solutions are diverse and complex and can be built in a heterogeneous nature due to the utilisation of multiple services that may originate from various cloud providers (e.g., multi-cloud applications). Such solutions' resilience can be measured by the persistence of dependability when facing changes [11].

Fault injection is a common method that is used to evaluate the dependability of software systems [8], which involves intentionally injecting faults or errors into the system to assess its behaviour and impact on quality metrics such as availability, reliability, resilience, and performance [7, 8, 12]. This approach can be applied during runtime, while the application is executing, or during compilation time, where faults are specified in the code prior to compilation [13]. Fault injection can also be organised as a fault injection campaign, where a set of observed or assumed faults are systemically injected to cover possible fault combinations comprehensively [14]. This technique has been utilised in analysing the performance of cloud-based software systems to assess their dependability and reliability [6, 12, 15]. In other words, it is the software equivalent of hardware stress testing, and it has the potential to minimise the uncertainty based on the idea that software systems will unavoidably experience both internal and external faults: A realistic "worst case" fault load is applied to the deployed system, and it should still maintain the intended degree of availability, performance, and service quality [15].

Chaos engineering [14] is the discipline of experimenting on a distributed system to build confidence in a system's capability to withstand unexpected failures. This method is used to improve complex systems' resilience by intentionally introducing faults and measuring the results. By simulating controlled failures, Chaos Engineering identifies and addresses potential system vulnerabilities and weaknesses in a system's design, architecture, and operational practices. There are many ways to inject failures into a system, including shutting down a service, injecting latency or errors (e.g. denial-of-service attack), terminating processes or tasks, or simulating a change in the environment or configuration setting. This facilitates the measurement of performance and dependability metrics such as system uptime, error rates, and recovery times. This helps in a better understanding of how the system responds to intentional fault introduction and improves the resilience of the systems using the obtained information. Thus, ensuring the dependability of cloud-native applications is challenging and needs more investigation.Chaos engineering simulates real-world conditions and failure scenarios to assess system resilience and dependability. Fault injection introduces faults into test systems, while chaos engineering advocates for the intentional injection of faults directly into production systems. This enables teams to identify weaknesses and improve overall system resilience.

The dependability and performance of cloud-native apps are closely associated with one another. Dependability is the capability of a system to avoid service failures, and it encompasses quality attributes like reliability, availability, safety, and security [7].

All of these characteristics are directly impacted by performance problems; for example, an application's dependability gets harmed when it continuously suffers from problems in performance standards (slow response times, frequent crashes, etc.). On the other hand, a dependable application usually keeps up consistent performance metrics. Considering Availability, poor performance may result in decreased availability. Users may view an application as unavailable if it becomes extremely slow or unresponsive, which could impact the service's overall availability. The same is true for safety and security, as performance issues may occasionally point to hidden security weaknesses or hazardous application behaviour. Thus,

dependability includes availability, reliability, security, safety, and problems with performance might threaten these attributes.

The software's level of dependability is assessed through many indicators, such as its reliability, which is its ability to consistently work over a predetermined period of time and under predetermined circumstances. Moreover, while evaluating the availability of the software, we evaluate how much time a software is running for a specific amount of time, and availability is considered an important indicator of dependability. Also, the system's ability to function without harming its users or the environment is how we measure safety, which is another indicator of dependability. As for security which is another indicator of dependability, we evaluate how well the system is protected against attacks, data breaches, and unauthorised access. Resiliency evaluation is another aspect of dependability, which is the ability of a system to recover and carry on in the face of malfunctions. Tracking performance metrics and their effect on dependability attributes has the potential to help establish the relationship between dependability and performance degradation. This also helps in understanding how the variations in performance impact the system's overall dependability, such as affecting availability by increased response time, which may indicate a growing dependability problem. Monitoring tools and observability norms are crucial for evaluating dependability and performance metrics in cloud-native applications. These tools aid in the detection of deviations, the identification of potential weaknesses, and the implementation of mitigation measures to guarantee that performance problems do not affect the application's dependability.

This empirical study investigates how well cloud-native applications can function and behave in the face of both anticipated and unanticipated events. In particular, we are trying to investigate how introducing various delay times impacts the performance and, accordingly, the dependability of the application. Thus, this paper conducts a number of controlled experiments to answer the following research questions:

- *RQ1* Can Chaos Engineering, specifically the practice of fault injection, be invested effectively to test and improve the dependability of cloud-native applications?
- *RQ2* How can fault injection be used to evaluate a system's behaviour under high loads or adverse conditions (such as injecting a delay and increasing the duration of the injected delay and the number of concurrent users)?
- *RQ3* What other problems are identified besides performance when a system experiences increased loads and adverse conditions?

Thus, the paper evaluates the potential of Chaos engineering and, specifically, the practice of fault injection in testing and improving the dependability of cloud-native applications. This answers the first research question. It also investigates how increasing the loads and the adverse conditions on a system can be used to evaluate the systems behaviour, which answers the second research question. Moreover, to answer the third research question, the paper investigates the complications other than performance, such as failure points and other dependability issues encountered when a system experiences increased loads and adverse conditions.

The remainder of the paper is structured as follows: Sect. 2 presents the fundamentals, research background and related works. Section 3 provides the exploratory Application-level Fault Injection Methodology. Section 4 presents the results of two case studies on cloud-native applications. This is followed by a discussion of the study in Sect. 5, including the implications, limitations, and importance of the work. The paper conclusions are given in Sect. 6, the last section.

2 Fundamentals, background, and related works

This section presents the fundamentals and research background with its related works for our study. We discuss the Cloud-Native applications and their dependability and how we can assess the dependability and performance of such applications using fault-injections and chaos engineering. Finally, we discuss how this will be important in terms of the availability of cloud-native applications.

2.1 Cloud-native applications and their dependability

Kratzke et al. [16] provide a definition for the term cloud-native application as a distributed, elastic, and horizontally scalable system made up of (micro)services that isolate the state in the fewest possible stateful components. They added that it is where the application and every standalone deployment unit of it is created using cloud-focused design patterns and run on an elastic self-service platform.

Cloud-native applications are innovative solutions for delivering on-demand services in a pay-as-you-go manner that assures substantial cost savings. However, these advantages come at the cost of the greater complexity of systems, raising new challenges in ensuring service dependability and resilience for applications running in a Cloud environment [17].

In general, any fault in any component of any system has a tendency to propagate to other components, cause components' malfunctions, and finally create failures [18]. Simulating failure uncovers the impact of any fault at a component level on other different system components. Cloud services are recursive in nature because different cloud services are interconnected and they rely on one another, accordingly, such cloud services can malfunction one after the other, causing a chain of failures across several clouds [19, 20]. Testing the dependability of cloud-native applications using chaos engineering will expose additional dependability issues in the system, including availability and reliability, in addition to testing and assessing the performance of such systems.

Cloud customers expect cloud providers to have strong processes in place to reduce or avoid cloud services failures. It is anticipated that such providers are responsible for maintaining an excellent degree of resilience and dependability, even in the event that a single component of the cloud infrastructure fails. Therefore, Cloud service providers focus on measuring and reducing their operational risk more as cloud operations expand. Performance and dependability metrics are included in this. While dependability metrics evaluate the reliability and

accessibility of cloud systems, performance metrics evaluate the effectiveness and track the performance of these systems. It is crucial to do both evaluations to ensure that cloud services meet customers' needs [19]. This research's major objective is to explore the dependability and performance issues of cloud computing. It discusses the challenges involved in developing metrics for assessing the elasticity aspects of performance in cloud-based systems and provides practical communications on comparing different cloud service providers. The research also examines the latest developments and cloud computing technologies that can have an impact on performance and dependability metrics.

To conclude, fault injection has been utilised in previous studies to measure dependability, yet those studies were either preliminary or not specifically focused on cloud applications. For example, Herscheid et al. [15] suggested a design architecture for a "fault injection as a service" within OpenStack, but the service implementation is still under development. A new fault injection architecture for system calls was introduced by Zhang et al. [21]. However, the study's objective was to evaluate application reliability in relation to system call invocation errors in real-world settings and did not specifically mention cloud-based apps.

2.2 Chaos engineering for cloud-native applications

Chaos engineering is a discipline that helps to identify and address potential weaknesses in complex systems by intentionally injecting faults or disturbances. It can be used as a systematic assessment approach for evaluating the dependability of a system. It has also examined cloud infrastructure resilience, and with its help, developers could be able to evaluate how random faults or delays would impact an application [22]. Moreover, chaos engineering enables enhancing confidence in software systems and in their ability to survive and stand in front of unexpected faults and errors by testing software systems in environments that are very similar to those used in reality.[1]

However, limited research has been done on employing chaos engineering to measure the performance of cloud-native applications. Chaos engineering is frequently used to test the security and resilience of cloud-native systems. According to related empirical research [23], performance evaluation of cloud-native applications is a common topic for papers. For instance, Eismann et al. [24] looked at the stability of performance testing for serverless applications. Additionally, Scheuner [25] presents an in-depth analysis of serverless applications and highlights the common characteristics and performance requirements associated with adopting these solutions. The study reviewed the performance of 112 Function-as-a-Service (FaaS) from academic and industrial sources, identifying 89 serverless applications that adhere to the principles of cloud experimentation's reproducibility. Thus, when it comes to employing chaos engineering to evaluate the performance of cloud-native apps, the reported works are still limited.

[1] http://principlesofchaos.org/.

Fault-injections have been used to evaluate the performance of cloud computing applications. Cerveira et al. [26] investigate the use of virtual machines in cloud computing and their susceptibility to soft errors by examining various failure modes of applications, operating systems, virtual machines, and hypervisors. A new framework [27] for examining the scalability of cloud-based applications is presented by a recent study that uses application-level fault injection. The experiments conducted for this study examine the delay latency as software faults in two distinct settings and two demand scenarios, demonstrating the impact of faults on the resilience and scalability of cloud applications. Along with these studies, other research has focused on evaluating the resilience of IT service configurations using chaos engineering principles and techniques [28], as well as evaluating Self-Adaptive and Self-Healing Systems [29].

Chaos engineering research has limited application in evaluating the performance of cloud-native applications. A recent research paper [4] stated that chaos engineering has a different impact on the system performance, where some measurements were collected to investigate the impact of injected faults on the overall system performance. However, the study focused on providing a chaos tool as a software product line to improve the systems' resiliency. Moreover, in a basic latency-injection configuration, an MSc thesis [30] investigated the resilience of Lambda functions when chaos engineering principles are applied. By focusing on the functions' cold-start and warm-start latency, the study examined the effect of latency injection on the functions' availability and resilience.

2.3 Availability in cloud computing

The use of online services that rely on high availability, which is considered a fundamental criterion, is increasing worldwide. Availability is a percentage-based indicator of how frequently consumers can access and use services [31].

With consumers, downtime with acceptable value should be established, and in the case of critical services, it might be necessary to provide a service with high availability and minimum downtime [32]. Thinking on the development and management of highly-available services is initiated by this. It is well recognised that CC may provide the necessary resources, and when operating in a cloud environment, providers agree to provide the requested services with the agreed quality metric value. Moreover, users expect cloud services to be largely available, and even a few outages may damage a cloud provider's reputation and financial position [19]. However, in reality, providing services suffers from problems that cause not providing the promised quality of service. Because of this, service users might get unsatisfied and decide not to use those services, putting the reputation of the service providers at risk [32].

Over the past two decades, cloud service consumers have become more interested in selecting cloud services through quality analysis. While many companies use enterprise cloud services to develop their cloud-based service systems or individual customers use apps on mobile devices, both of them face a challenge at the selection time as different vendors provide similar functional properties (services),

and of course, both of them seek out relevant, reliable, adequate information and benchmarks to assess cloud services [33]. Having such details facilitates selecting the ideal service. Moreover, service consumers always review and rate the provided services, which enables other consumers to select the most dependable services.

Cloud service providers may be able to provide better services if they are given the tools to evaluate the services they offer and investigate the impact of various faults on the dependability of the applications that use those services. This is solely due to the fact that such service providers will be able to enhance the offered services and understand how resilient such services are. Thus, it is necessary to explore the effects of various faults on cloud-native applications while they are in operation because this could reveal issues that conventional dependability techniques might not be able to detect.

3 Exploratory application-level fault injection meethodology

This study establishes an exploratory methodology for examining the dependability of cloud-based software services. The established method can be employed to look into how runtime fault injection at the application level affects how the application behaves in terms of certain dependability aspects. This could aid in facilitating the development of specialised protection services that assist cloud applications in order to deal with faults that might propagate through software components and result in failures that would impair the dependability of the software. The goal here is to explore and determine when the application begins to experience issues and, if it crashes, when it does so. This occurs as a result of a growing demand for services to be offered without interruption. Such tests assist in locating the point at which the application begins to suffer and crashes and assist in preventing reaching this point of failure, which, if it has occurred, results in users' dissatisfaction.

Chaos engineering involves intentionally inducing failures in a controlled environment to identify and resolve system issues and gain new insights about the system. On the other hand, stress testing evaluates a system's behaviour under high loads or adverse conditions to identify whether they propagate to cause failure points and assess the system's robustness. Our testing methodology begins by injecting faults into the system's baseline state. This helps us understand how the system behaves under stressful scenarios. We gradually increase the intensity of the fault injections until we reach the point where the system stops responding (critical failure). This helps us determine the system's maximum stress point in its baseline state. In the context of HTTP-based applications like serverless or cloud-native applications, it is essential to ensure that they can withstand potential issues, including high response times. Therefore, the concern about response times aligns with the objectives of chaos engineering, which emphasizes the importance of testing and improving system resilience.

We established an exploratory methodology because our main goal is to investigate the complex dynamics of possible critical faults and how they spread to cause system failures. By utilizing fault injection, our method goes beyond conventional dependability analysis. Our goal is to find deficiencies and unpredicted

dependencies that may not be seen through conventional dependability analysis methods, which depend on simulating real-world scenarios. This level of in-depth investigation of real-world scenarios instead of simulating them helps us identify the different ways in which critical faults propagate inside the system.

Although traditional dependability analyses are certainly capable of identifying critical faults, we think that our special emphasis on fault injection and actual application scenarios offers a more complex and thorough insight. We believe that conventional methods could be able to find critical faults. However, its application to real-cloud applications, such as microservices within cloud-native apps and serverless architectures, has significant challenges that require an exploratory methodology. Because these environments are dynamic, we have to adapt and expand on conventional methods to maintain their applicability and usefulness. Thus, adopting an exploratory methodology is motivated by the need to thoroughly comprehend the dynamics of critical fault propagation within the distinctive environments of serverless architectures and microservices. We think that our exploratory methodology increases the understanding of system dependability in modern cloud-native contexts and offers insightful new information to the area. The following are the main steps of the exploratory methodology:

- *Identification of Dependability Goals:* The dependability goals for any system should be identified by identifying the most critical quality attributes. This has the potential to help in identifying the most critical failures that are to be analysed in the next step.For instance, in the case that availability is chosen as the dependability goal, every delay that leads to failure in service is seen as a critical failure, and the fault that may cause it is regarded as a critical fault).
- *Critical Failures and Critical Components:* It is crucial to identify the most critical failures because the fault injection technique is expensive and should be used carefully. Following the analysis of these specified failures using various mature and well-established failure analysis tools and techniques such as (FMEA) and (FTA), representative fault repositories representing potential real faults and inputs that may result in such critical failures that the system may encounter during operation are approved. The system's critical components or subsystems are those that could be negatively impacted by these flaws and whose failure would cause the entire system to fail. Critical components must be assessed for dependability, and they should be identified because they might make the system fail if they encounter any of the possible flaws.
- *Fault Injection Scenarios:*The most critical scenarios or failure modes relevant to the system under assessment are chosen. These may include failures due to network faults, faulty hardware, software faults, and other potential disruptions that the system may encounter in real-world scenarios.
- *Plan experiments (Choosing fault injection techniques and tools):* Controlled experiments are planned and designed to simulate the identified failure scenarios. These experiments should be carefully designed to minimise risks, ensure safety, and align with the defined scope and objectives. Moreover, Fault Injection techniques and tools should be selected at this stage.

- *Configuration and Execution of Fault Injection:* A number of controlled experiments should be executed to uncover unpredictable behaviour. These experiments may involve injecting failures into the system, such as shutting down servers, injecting delay latency, or generating faults in software components. The system's behaviour, especially in terms of performance, should be carefully monitored and measured during the experiments, utilising appropriate monitoring tools and techniques that enable the collection and analysis of data. Relevant metrics, such as response times, latency, throughput, error rates, and system availability, are captured to assess the system's dependability. Thus, the results are analysed at this stage to identify the weaknesses in the application's resiliency and how they can affect the application's functionality.
- *Improvement proposals and iterative testing:* After identifying the vulnerabilities, weaknesses, and areas for improvement in the system's dependability, potential issues that may not be apparent in traditional dependability analysis are uncovered. Repeating the testing process with different scenarios helps to approve and generalise the results. Based on the findings from the chaos engineering experiments, appropriate remediation plans should be developed and implemented to address the identified weaknesses. This may involve changing the system architecture, improving error-handling mechanisms, or implementing redundancy mechanisms. Thus, the process should be iterated by repeating the experiments to validate the effectiveness of the remediation efforts.
- *Automating the Resiliency Testing:* To enable ongoing and routine resilience tests, the fault injection procedure can be integrated into the deployment process.

By following these steps, we can effectively utilise chaos engineering as a systematic assessment approach to evaluate the dependability of complex systems, identify vulnerabilities, and improve the overall resilience and reliability of the system.

4 Case studies on cloud native computing and chaos engineering

Our methodology is applied to measure the impact of chaos engineering on cloud-native applications. We performed experiments on Amazon AWS using two cloud-native architectures, serverless and containerised microservices. We have used two types of fault injections and demonstrated our methodology using two open-access cloud-native applications. All experiments setups were connected to load-balancing services, a cloud-native fault-tolerance mechanism, to reduce the risk of performance issues in the applications [34]. The first case study presents 480 experiments using a serverless application and delay injections within the Lambda functions. For the second case study with 280 experiments, we examined a containerized microservices application and introduced fault injection through slower connection speeds to test the application's performance under these conditions.

Table 1 AWS technical settings for leave management application

Item name	Value
CPU	8vCPU
Memory	24GB
DynamoDB version	DynamoDBv2
AWS services	Amazon elastic container service (ECS)
	Amazon elastic container registry (ECR)
	AWS Lambda
	Amazon elastic load balancer (ELB)
	Amazon simple notification service (SNS)
	AWS X-ray
	AWS CloudWatch
	Amazon API gateway
	Amazon DynamoDB

4.1 Case study 1: serverless computing and chaos engineering

Serverless architecture has been growing in popularity in recent years, and many organisations and companies have started using this cloud-native option to operate everyday applications. In a serverless environment, developers may focus on developing code instead of managing the underlying cloud infrastructure. Serverless options are available through public cloud services, and they combine two key components, backend-as-a-service (e.g., managed storage and databases like Amazon DynamoDB and Relational Database Service) and function-as-a-service (e.g., AWS Lambda, Microsoft Azure Functions) [24]. The serverless functions are run as event-driven execution in isolated containers on demand. These functions are triggered to respond to any event, like an HTTP request.

In this paper, multiple controlled experiments were conducted following the methodology explained in Sect. 3. We used a serverless, cloud-native application called Leave Management Application to conduct experiments on the Amazon AWS cloud platform. This serverless solution is an open-source .NET application based on decoupled architecture design principles. The application was built using native AWS services and published on GitHub[2] by a Specialist Solutions Architect at AWS. This container-based application was hosted using the AWS services and technical settings, as indicated in Table 1.

The architecture of the serverless application, as shown in Fig. 1, depends on utilising and incorporating cloud-native services within the application. The application is a human resource leave management system with two main functionalities: submitting leave requests and deciding leave requests. GetLeaveRequest and AddLeaveRequest both follow the GET and POST HTTP functions, respectively.

[2] https://github.com/aws-samples/aws-serverless-workshop-decoupled-architecture.

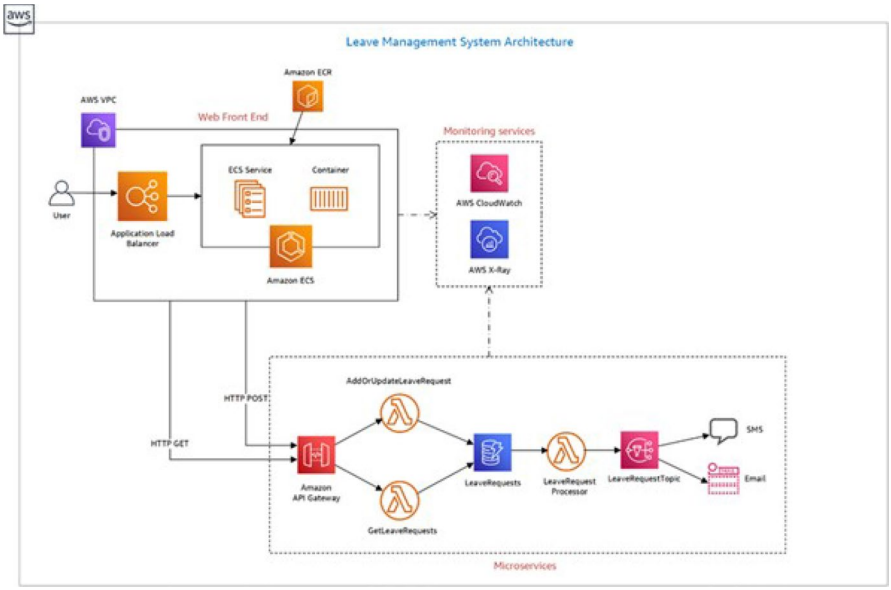


Fig. 1 Leave management application architecture (adopted from the application Github page)



Fig. 2 Service map using AWS X-ray the Lambda functions

The AWS application starts working when HTTP requests arrive, which are handled by the application Load balancer (ALB) to distribute traffic to multiple servers. The API Gateway redirects requests to the appropriate lambda function, such as GetLeaveRequest or AddLeaveRequest, which connects to the DynamoDB NoSQL database for data storage and retrieval. AWS Cloud Watch and AWS X-Ray are used to monitor and troubleshoot application performance and errors. AWS X-Ray traces requests through the application and identifies performance bottlenecks. The Leave application has two main Lambda functions: Request Flow and Get Leave Request. Any HTTP request Flow saves leave

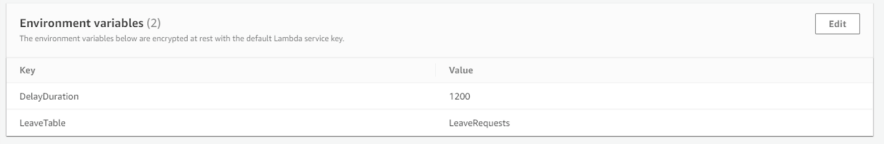


Fig. 3 Delay duration environment variable

requests in DynamoDB, while Get Leave Request retrieves leave requests from the same table.

Figure 2 which was generated using the AWS X-Ray service, displays the service map at runtime. This confirms that the application works correctly and that HTTP requests are appropriately linked to their respective cloud services.

4.1.1 The application-level fault injection implementation

The application-level fault injection methodology presented in Sect. 3 is applied to the leave management application.

1. The main dependability goals are specified:
   - There should be no significant delays in the system's operation because users may suffer severe consequences as a result of high latency.
   - There should not be any downtime for the system or service failure. One significant failure that could have serious consequences is if the service is unavailable.
2. Critical failures and critical components are identified. Since any faulty input (wrong data, late data, or missing data) for the Lambda function has the potential to propagate and hinder the system from providing service, the lambda function has been identified as a critical component, and the delay has been identified as a critical fault.
3. A fault injection scenario is identified. A delay of 300, 600, 1200, 10,000, and 27,500 milliseconds (ms) is to be injected inside the deployed Lambda functions. In this case study, the system collapsed when we started injecting a delay of more than 28,000 ms.
4. The controlled experiment is planned and designed to stimulate the identified failure scenario.
5. Configuration and execution of fault injection. Chaos engineering was implemented inside the deployed Lambda function by setting up an environment variable in AWS to delay the duration (in Milliseconds), which was then injected into Lambda functions, as shown in Fig. 3.

As a controlled experiment to determine the impact of the injected delay on the systems, introducing a delay to the serverless function is a form of chaos engineering. The performance and resilience of the system may be impacted by this delay both

positively and negatively. On the one hand, the delay may mimic a real-world scenario in which the function takes longer to run than anticipated. This can lead to improvements in resilience by pointing out potential bottlenecks or issues with the application's ability to handle delays. Also, the additional delay can have a negative effect on the system's performance by slowing down the function's overall execution. Despite this, it is possible to view the adverse impact on performance as a trade-off for a positive impact on resilience. Then, a const string is declared to hold the name of the environment variable by

```
const string
DelayDuration_ENVIRONMENT_VARIABLE_LOOKUP = "DelayDuration";
```

After that, the snippet is injected, causing the thread sleep to be triggered for the functions Get Leave Request and Add Leave Request, as seen in the following codes. These codes are injected into the application's codebase.

```
For the Add Leave function:

int DelayDuration = Convert.ToInt32
(System.Environment.GetEnvironmentVariable
(DelayDuration_ENVIRONMENT_VARIABLE_LOOKUP));
//var leave=JsonConvert.DeserializeObject<Leave> (request?.Body);
      if (DelayDuration > 0)
      { System.Threading.Thread.Sleep(DelayDuration); }
For the Get Leave function:

int DelayDuration = Convert.ToInt32
(System.Environment.GetEnvironmentVariable
(DelayDuration_ENVIRONMENT_VARIABLE_LOOKUP));
   if (DelayDuration > 0)
     {System.Threading.Thread.Sleep(DelayDuration);}
```

This will follow by publishing this code using the command line (cmd) to replace the old Lambda hosted on AWS. Then, the modified Lambda functions are uploaded to update the application code in the AWS container.

6. Data collection was then done using monitoring tools and techniques. Performance metrics such as throughput, latency, and error rates were recorded during the experiments. The number of requests per second (throughput) is identified, as well as the latency, which is measured in milliseconds (ms). The error rates, which are identified as a percentage of the total requests, are also measured.
7. The delay conditions were applied in multiple iterations to account for the variability. The system was monitored continuously, and the metrics were collected through iterations. Each experiment is repeated ten times.

Table 2 Experiments duration (total time ×10 times)

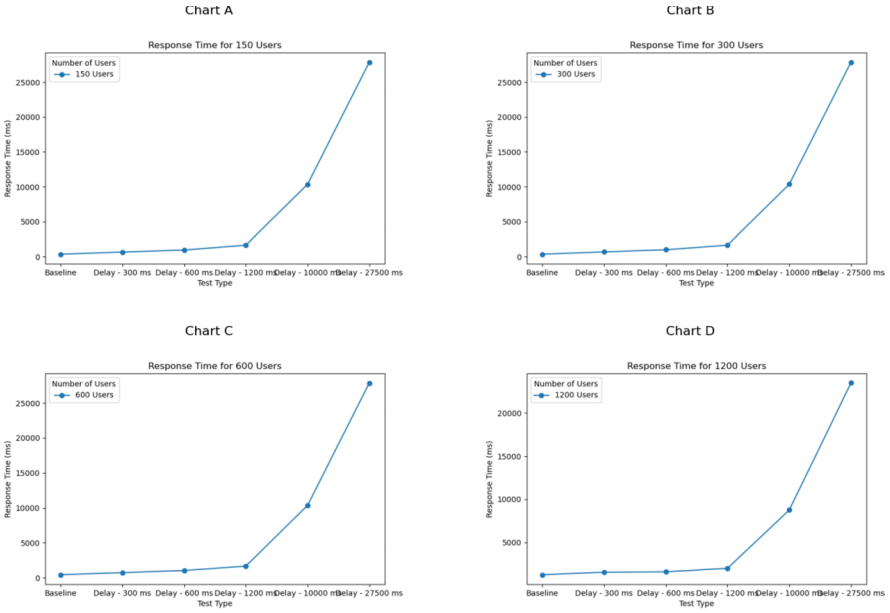
Users	Baseline	300 ms delay (s)	600 ms delay (s)	1200 ms delay (s)	10,000 ms delay (s)	27,000 ms delay(s)
Get API						
150	15	20	21	28	160	290
300	15	20	21	28	160	290
600	20	25	26	33	165	295
1200	20	25	26	33	165	295
Add API						
150	15	20	21	28	160	290
300	15	20	21	28	160	290
600	20	25	26	33	165	295
1200	20	25	26	33	165	295

4.1.2 Testing methodology and process

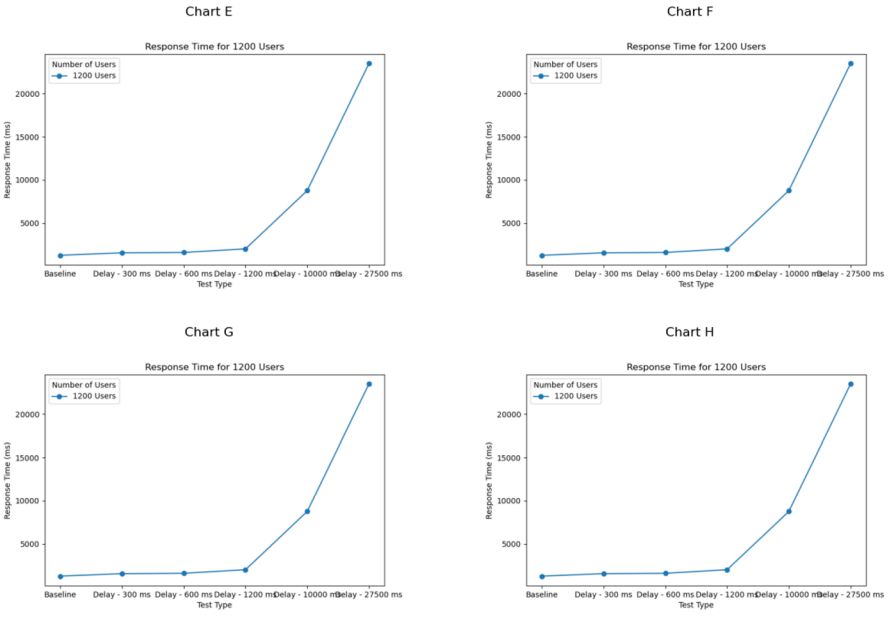
The study established a testing process and approach to determine how chaos engineering affects the dependability of the serverless application following the methodology presented in Sect. 3. This test methodology was executed using the JMeter testing tool. Firstly, the test methodology is conducted on the Serverless application without chaos engineering, capturing and analysing the results as our baseline benchmark. Next, the same test plan and parameters were performed on the Serverless application with chaos engineering with injected delay 300, 600, 1200, 10,000 and 27,500 ms, capturing and analysing the results. Then, both groups of results were compared to measure the impact of chaos engineering on the dependability of Serverless applications.

Performance metrics were collected using the same experimental setting for both the baseline and the chaos engineering experiments to ensure the experiments' integrity. Response Time, Latency, Throughput, Lambda Duration Minimum, Lambda Duration Average, and Lambda Duration Maximum are among the collected metrics. We adopted the definitions of these metrics from JMeter and AWS Lambda Monitoring and captured these metrics from both the JMeter Testing tool and AWS Cloud Watch service.

Two Lambda functions were tested within the application, using four different numbers of users incrementally: 150, 300, 600, and 1200 users. Each experiment was repeated ten times to ensure statistical significance. The averages over the ten runs were calculated for each test and noted that the standard deviations were small relative to the averages. Each Lambda test was conducted on the same day to ensure the stability of the Lambda execution and results. This is to ensure that we have reliable and stable data, as recommended by [24]. Our test plan included two Lambda functions: Get Leave Request and Add Leave Request. We used different thread groups for each test. This paper reports a result and analysis of a total of 480 experiments, as shown in Table 2.



(a) Get Leave Request



(b) Add Leave Request

Fig. 4 Results for response times for Lambda functions on Amazon AWS EC2

4.1.3 Case study 1: results and discussion

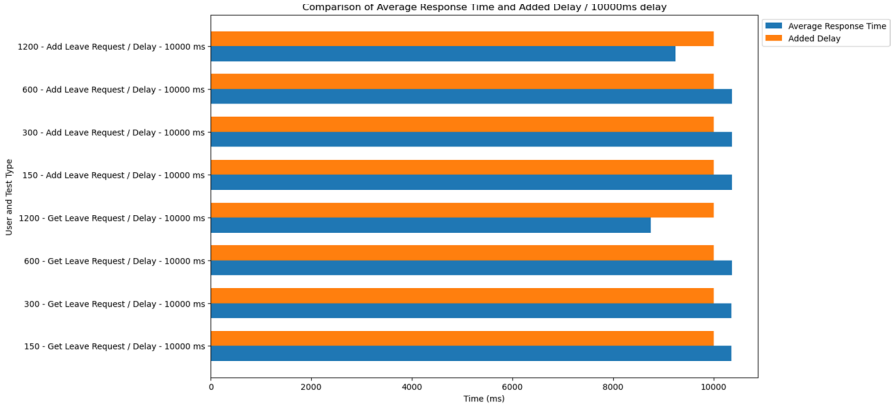
This exploratory empirical study revealed the following results for the Serverless application:

Impact of delay on performance Figure 4 shows the average response time for ten experiments for each different delay period and a certain number of users. The average response times of the Get Leave Request for every experiment for each injected delay and the load of 150 users are shown in Fig. 4 A. The resulting pattern shows a clear increase in the response time when the injected delay is increased, which makes it logical given the injected delay. For example, the application was unable to handle injecting a 300 ms into the lambda function. This is evidenced by comparing the baseline response time of 378.09 ms (without adding any delays) with the response time when injecting a 300 ms delay into the Lambda function, where a delay of 300 ms in addition to an unexpected additional 4 ms of response time occurred, bringing the response time to 682.82 ms. However, it also shows that the response time not only increased by the amount of the injected delay, it did so plus an additional 4 ms. When a 600 ms delay is injected, the average response time in the ten experiments is observed to increase to 969.5 ms, which went up by the inserted delay minus nine seconds. After injecting 1200 ms, a response time of 1658 ms was recorded, which is an increase of 1306.71 ms from the baseline. In other words, we noticed that the system is starting to experience response times greater than the injected delays. This system behaviour was pretty clear when a delay of 27,500 ms was injected; at that point, the response time changed to 27874.5, indicating that an additional 27.49 ms had appeared out of nowhere. In other words, we recorded a pattern of increased response time for the Get Leave Request when increasing the injected delay for the same group of users.

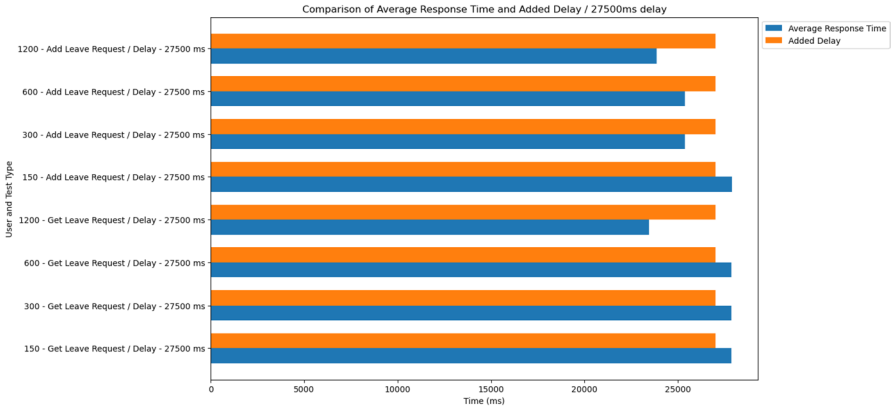
Figure 4B shows that the same pattern was observed while conducting the same experiment with the load of 300 users ten times for each injected delay period. The response time increases when the injected delay increases. However, we also found that there was extra time that was injected, which is even greater than the additional time when having only 150 users. For example, when having 300 users, the average response time was 708.04 ms after injecting 300 ms, indicating an additional 25.57 ms of delay. This increase over the inserted delay in the case of 150 users was only 4 ms. When injecting 600 ms, 1200 ms, or 27,500, the same effect is recorded with more latency added on top of the injected time than when there are 150 users compared to 300 users.

Figures 4C shows the average recorded response times from the ten experiments for each delay period with a load of 600 users. For the 600 users, injecting 300 ms and 600 ms is found to increase the response time with a period less than the injected delay. However, there is an increase in the response time, which is higher than the injected time while injecting 1200 ms and 27,500 ms, which calls for justification.

The averages of response time for the ten experiments with each delay for 1200 users are shown in Fig. 4D. When injecting 300 ms, the response time for the Get Leave Request deviates from the baseline, but only by 286.62 ms, or 13.38 ms less than the injected delay. The average response time when injecting a 600 ms delay for the same number of users was 1608, which implies that it varied from the baseline



(a) 10000ms Delay



(b) 27500ms Delay

Fig. 5 Results for response times for Lambda functions on Amazon AWS EC2

by just 327.41 ms, which is less than the injected delay of 600 ms. A response time of 2019.5 ms is recorded while injecting 1200 ms, and there are 1200 users, compared to 1280.59 ms for 1200 users without injecting any delays. In other words, the response time increased by 819.5 ms when a 1200 ms delay was inserted, which is less than the injected delay time. At this point, a 3.58% error rate is recorded. Therefore, approximately 43 service consumers are estimated to have been left without a service. At this point, we recognized that we were starting to experience problems with availability in addition to performance.

Another odd behaviour was observed while serving more than 1000 consumers, introducing a delay of 27,500 milliseconds. Any number after 1000 people was served normally every time the experiment was run with this number of users and this particular delay; however, after that, the application experienced the delay, and the latency only affected the other 1000 users. For instance, when there were 1200 users, the first 200 were served. After a delay, the remaining 1000 users began to

be served and were impacted by the latency. Thus, response time and error rates dramatically increased after the injected delay time was increased to 10,000 ms and 27,500 ms.

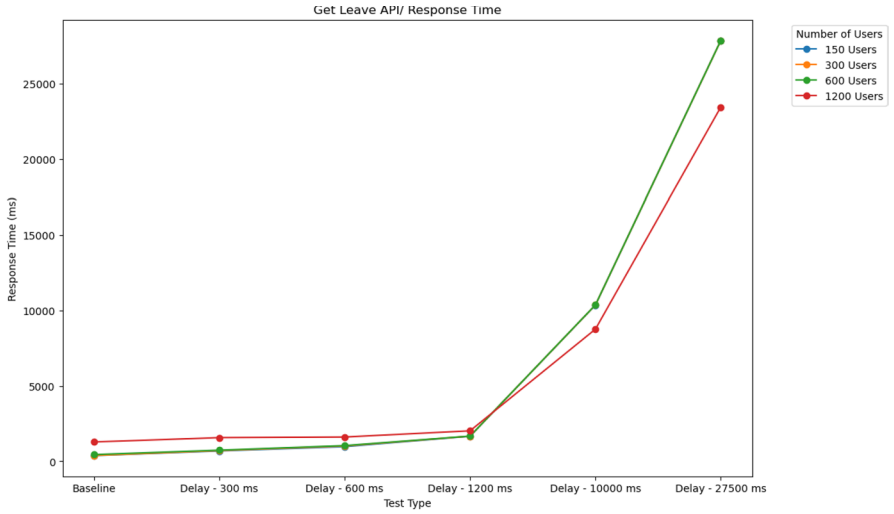
Figure 5 shows that there is an obvious correlation between the injected delay and latency since the latency rises as the delay does as a result of a longer response time. When the number of users is increased once more, the Lambda service receives more HTTP requests to run functions via the Lambda API. As the Lambda response time increased, the latency for the same delay period increased. Both "get leave request" and "add leave request" have the same relationship, that in both of them, the latency increases while the number of users increases for the same injected delay period. Thus, increasing the number of users for the same injected delay period has an adverse effect on performance.

In terms of the delay that happened, there is a difference in the behaviour between the get leave and the add leave. In other words, in the get leave delay, for all user groups, it can be seen that the get leave consumed the injected delay plus an additional time except for 1200 users when injecting 27,500 ms delay period where the average latency was 23444.8. The response time was less than the injected delay, although the Lambda function itself is run, and the recorded Lambda duration average was 27528.8 ms. In the case of the 10,000 ms delay, we note similar behaviour as both the Add leave and Get leave functions received less response time than the actual injections, which also contradicts the other user groups' behaviour. This can be seen in the Fig. 4 diagrams, which show the response times for Lambda functions for different user groups and different injected delay periods.

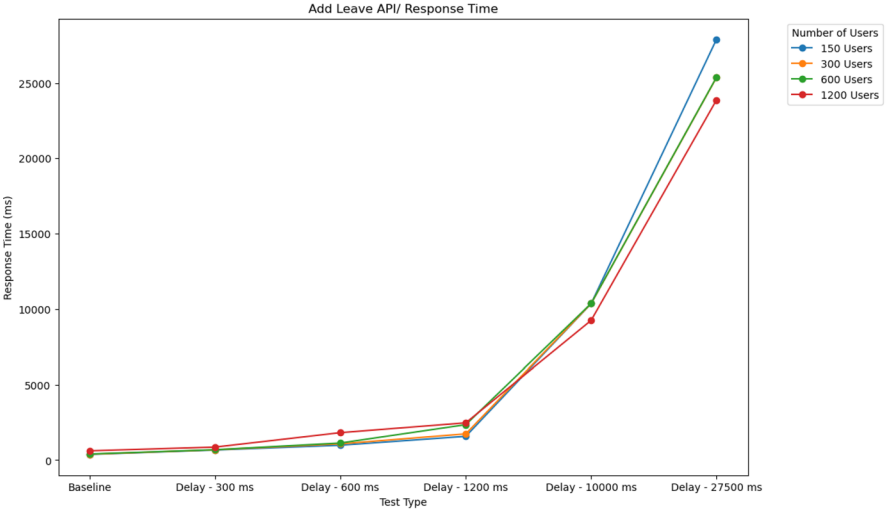
However, the "Add leave request" showed an entirely distinct behaviour in terms of response time (see Fig. 4), with almost all user groups (except for 150 users, which is a relatively small number of users) reporting delays more minor than the injected delays when 27,500 ms were used as the injection time. These noticeable differences between the GET and POST HTTP functions can be due to their intrinsic nature. POST (i.e. Add leave request) requests are always slower because encryption and decryption take place during the request and response. Additionally, the data is transferred in a single packet in GET requests, whereas in POST requests, the headers and body are transmitted separately. However, by conducting each experiment 10 times and calculating the average delay time for every single experiment, we verified the reliability of this finding.

Figure 6 shows the response time behaviour of all the experiments (A: Add Leave API and B: Get Leave API). Based on the graph, it is evident that all three user groups (150, 300, and 600) have a similar response time behaviour for the Get Leave API. However, the 1200 user group behaves differently after higher delay injection (i.e., 10,000 and 27,500 ms). On the contrary, the Add Leave API behaves differently as 150 user groups perform better; the 300, 600, and 1200 user groups were similar and have dropped in performance behaviour after higher delay injection.

Figure 7 illustrates the two Lambda function durations for all of the experiments. These results may be compared to the results of the Lambda function, showing that the Lambda function, as shown in Fig. 7 consumed the injected delay of 10,000 ms and 27,500 ms for all user groups, indicating that it already ran while users were still waiting for a response and paying for a service (or the business have been charged



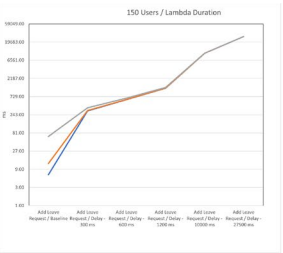
(a) Get Leave API



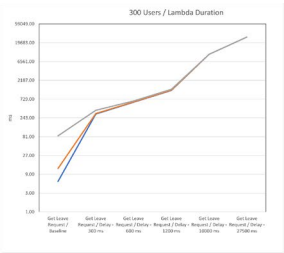
(b) Add Leave API

Fig. 6 Behaviour for response times for Lambda functions on Amazon AWS EC2

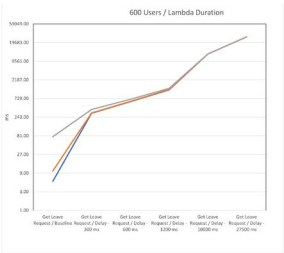
for the processing the requests) they weren't receiving. The Lambda function was consequently carried out, which was a costly operation because it required paying for the total runtime multiplied by the memory allocation. In other words, the service consumers are wasting their time and money. In terms of the 10,000 ms delay experiment, the response time increased in 150, 300, and 600 users experiments; however, it follows the behaviour of the 27,500 experiments when it reaches 1200 users. Unexpectedly, the response time dropped below the injected value (10,000



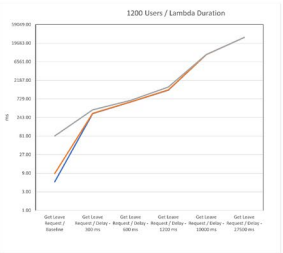
(a) Get Leave Request/150 Users



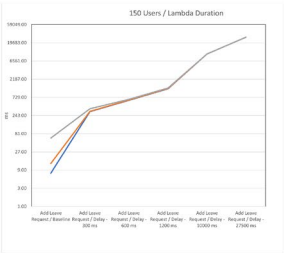
(b) Get Leave Request/300 Users



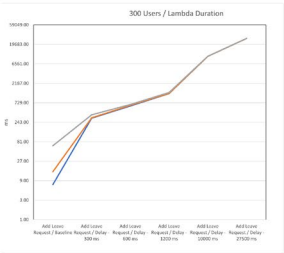
(c) Get Leave Request/600 Users



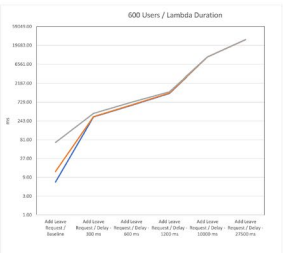
(d) Get Leave Request/1200 Users



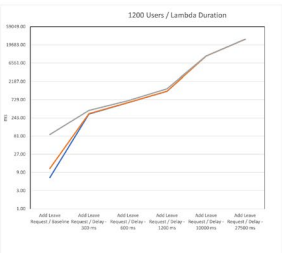
(e) Add Leave Request/150 Users



(f) Add Leave Request/300 Users



(g) Add Leave Request/600 Users

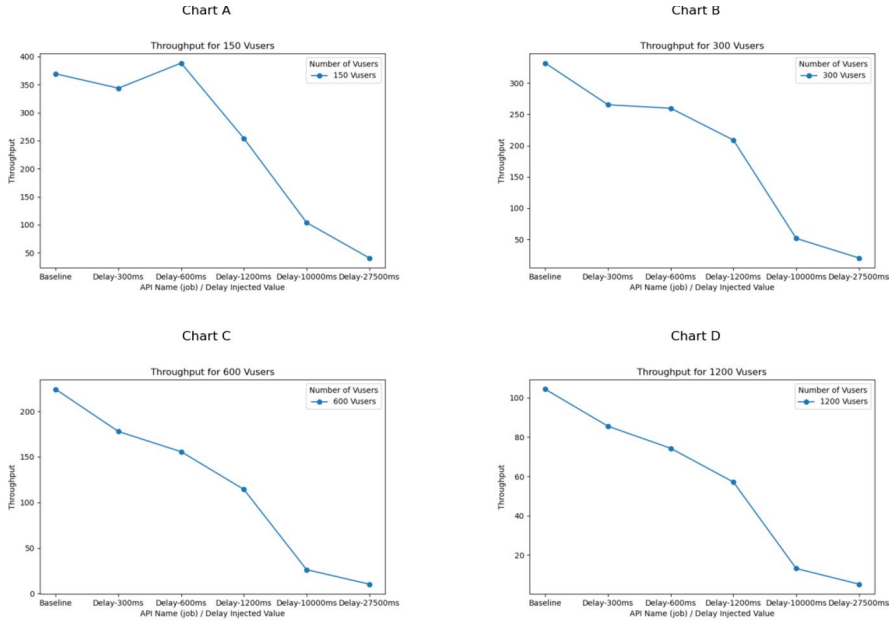


(h) Add Leave Request/1200 Users

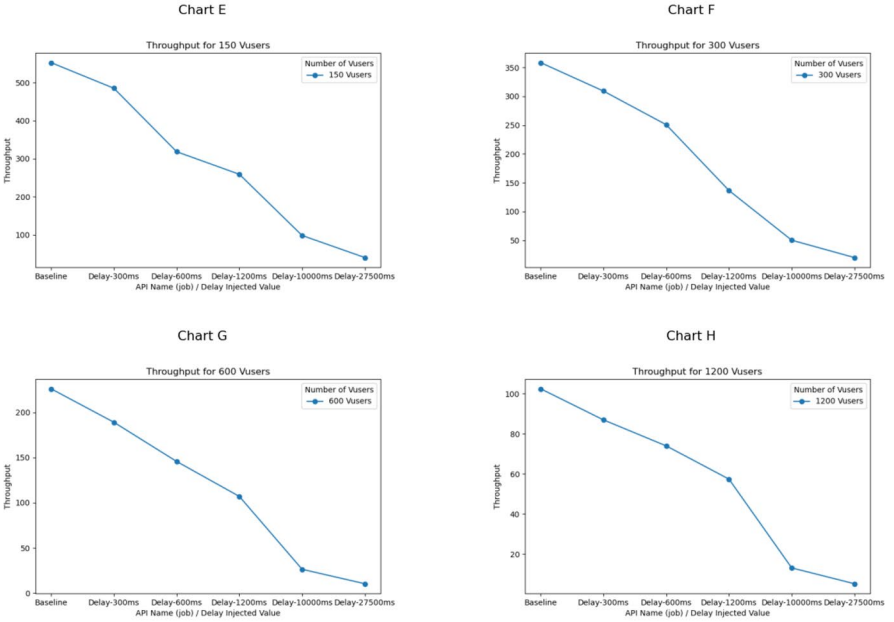
Fig. 7 Results for Lambda functions duration on Amazon AWS EC2

ms). It recorded 8758.53 ms and 9248.98 ms for the Get Leave Request and Add Leave Request experiments, respectively. In the contract, we can observe that the Lambda functions received a total of 10,000 ms injections, as shown in Fig. 7.

Figure 8 shows the average throughput for ten experiments with ten different delay periods for each number of users for the Get Leave Request. When injecting a 300 ms delay, we noticed a clear drop in throughput, that it dropped by 20% to 85.54 from 104.35 when no delay is injected for the same number of users when the sample size is 150. Here, we noticed the recurrence of this pattern over other delay periods for the same number of users. It is obvious that each user group has its unique trend of throughput decline as the injected delay increases. For example, a throughput of 343.74 was recorded when the number of users was 1200 users, and a delay of 300 ms was injected, which is a drop of 7% from the baseline where the recorded throughput for the same number of users without injecting delays was 369.37. This



(a) Get Leave Request



(b) Add Leave Request

Fig. 8 Results for throughput for Lambda functions on Amazon AWS EC2

drop in throughput continues for the other different delay periods (600 ms, 1200 ms, and 10,000 ms). This confirms the decreasing pattern of throughput for the Get Leave Request when increasing the delay period for the same number of users.

When injecting various delay durations, the Add Leave Request recorded an obvious decrease in throughput for the various groups of users. In other words, the throughput dropped by 18% while injecting 300 ms, and there were 150 users, and the drop was 20% for 300 users with the same injection of a delay. When there were 600 users and 1200 users, the throughput dropped for the same delay period, falling by 16% for the former and 14% for the latter. Additionally, it was noted that the throughput decreased further when a 600 ms delay was injected, increasing the drop in throughput to 74% for 1200 users. When injecting for 1200 ms, this decline rose progressively until it reached 113% for 1200 users. When there were 1200 users and a 27,500 ms injection, the throughput loss peaked at 1282%, which was a result that we could not explain. At this point, the latency also decreased by 97%, and a response time of 23870.48 ms was recorded, which is less than the injected delay period, as was previously mentioned. Table 3 presents the results of the performance indicators, including the calculated performance drops compared to the baseline. Such reduction in throughput and increase in response time is evidence that performance suffers when a delay period is injected into the critical Lambda function component. When both the user load and the injection time period are increased, the performance decline gets worse.

Availability Issues The system did not experience any availability issues with the 10,000 ms delay with 150, 300, and 600 users. However, unexpectedly, the application began experiencing serious availability issues and failures, with a 10,000 ms delay with 1200 users and 27,500 ms with 300 users. 17% of the requests in the 10,000 ms delay experiments were not met by the system, meaning that 170 requests were not granted any service to the original requests. When the number of users in the 27,500 ms experiments increased to 600, these issues were increasing and affected 0.5% of the users who did not receive a service. These problems worsened to affect 1% of the users when the number of users increased to 1200 users. As mentioned before, this is 17% of the 1000 users who are not served in terms of the 27,500 ms experiments. Eventually, the application crashes at a delay of 28,000 ms with any number of users. The system's unanticipated problems, which affected a high percentage of users, can be linked to the combined impact of delays and concurrent users. The system may experience outages and decreased availability when its performance deteriorates to the point where it becomes increasingly difficult for it to handle the load. This can, therefore, have negative effects on the application's reputation, user satisfaction, and financial performance.

Table 4 presents the error percentage recorded for the GET function, taking into consideration that POST never recorded any errors. In other words, JMeter did not record any errors for POST HTTP requests (i.e. Add leave request). However, the Lambda duration analysis (see Fig. 7) confirms that the system has processed the POST HTTP requests in all user groups and injected faults. This can be due to the intrinsic nature of POST requests, as they are slower due to the encryption and decryption that occur during the request and response. Additionally, the POST data headers and body are transmitted separately.

Table 3 Drop-in performance compared to the baseline—case study 1

Lambda function/delay	No. of users	Drop-in response time (%)	Drop-in throughput (%)
Get leave request/delay 300 ms	150	45	22
	300	46	26
	600	40	25
	1200	18	7
Get leave request/delay 600 ms	150	61	40
	300	62	44
	600	57	28
	1200	20	5
Get leave request/delay 1200 ms	150	77	83
	300	77	96
	600	73	59
	1200	37	45
Get leave request/delay 10,000 ms	150	96	138
	300	96	334
	600	96	1164
	1200	85	2713
Get leave request/delay 27,500 ms	150	99	1918
	300	99	2061
	600	98	1526
	1200	95	804
Add leave request/delay 300 ms	150	43	18
	300	43	20
	600	41	16
	1200	28	14
Add leave request/delay 600 ms	150	62	38
	300	64	55
	600	64	43
	1200	66	74
Add leave request/delay 1200 ms	150	76	79
	300	78	111
	600	83	162
	1200	75	113
Add leave request/delay 10,000 ms	150	96	113
	300	96	349
	600	96	1263
	1200	93	1183
Add leave request/delay 27,500 ms	150	99	1884
	300	98	2085
	600	98	1676
	1200	97	1282

Table 4 Error percentage recorded for case study 1

Lambda function/delay	No. of running users	Error percentage (%)
Get leave request/delay—1200 ms	1200	3.58
Get leave request/delay—10,000 ms	1200	17.00
Get leave request/delay—27,500 ms	1200	17.00
Get leave request/delay—27,500 ms	600	0.5
Get leave request/delay—27,500 ms	300	0.33

Table 5 AWS technical settings for PrestaShop application

Item name	Value
CPU	4vCPU
Memory	8GB
DynamoDB version	PostgreSQL
AWS services	Amazon elastic container service (ECS)
	Amazon elastic container registry (ECR)
	AWS Lambda
	Amazon elastic load balancer (ELB)
	AWS fargate
	AWS CloudWatch
	Amazon API gateway
	Amazon relational database service (RDS)

4.2 Case study 2: microservices computing and chaos engineering

The microservice architecture allows an application to be developed as a group of independent and autonomous services. These services are small, self-sufficient, and can be deployed separately. Microservices have gained popularity due to their ability to design complex systems consisting of loosely coupled and fine-grained software artifacts. Each microservice performs a specific function related to the application's business logic. Communication between microservices can be established through an HTTP resource API and message brokers, and they can be deployed in a lightweight container or virtual machine [35]. In this case study, we evaluate a cloud-native architecture that uses microservices running in Docker containers.

This paper conducted multiple controlled experiments following the methodology explained in Sect. 3. We used a microservices, cloud-native application called PrestaShop to conduct experiments on the Amazon AWS cloud platform. This container microservices is an open-source PHP application with a MySQL database based on docker architecture principles. The application was built using native AWS services, and this container-based application was hosted using the technical settings, as presented in Table 5.This universal open-source application is designed to build an e-commerce solution, with nearly 300,000 websites using this solution in

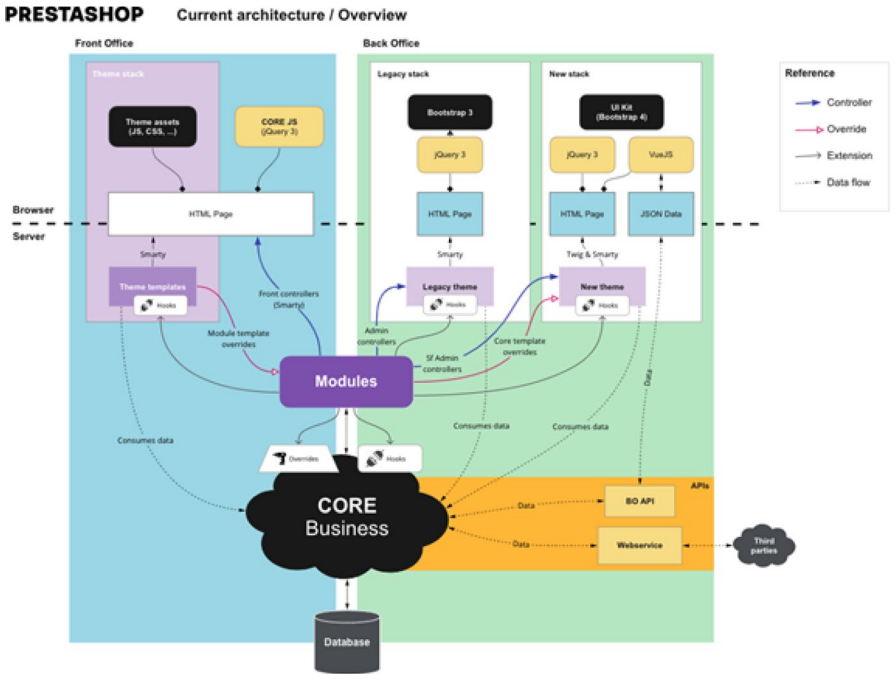


Fig. 9 PrestaShop application architecture [https://devdocs.prestashop-project.org/8/development/architecture/introduction/]

Europe and Latin America. Figure 9 presents the architecture of the application utilising cloud services and capabilities[3]

4.2.1 The application-level fault injection implementation

We used the fault injection methodology described in [27], which utilises generating slower connections to the system under test. We used Charles version 4.6.5, an HTTP proxy, monitor, reverse proxy and web traffic simulator, to simulate the application's slower connection in terms of delay latency (in milliseconds [ms]). The latency delay mimics the delay experienced on slower connections, which is the gap between sending an HTTP request(s) from the application side and receiving the request at the cloud server side. The delay latency times in the experiments in this case study ranged from 300, 600, 1200, 2400, 4800, and 15,000 ms. In this case study, as soon as we began introducing a delay latency greater than 15,000 ms, the system crashed.

In the chaos engineering experiments, we are testing for a delayed latency or the delay in the time it takes for a request to be made and received at the other end. This is tested just before executing the HTTP request against the targeted cloud

[3] https://devdocs.prestashop-project.org/8/development/architecture/introduction/.

Table 6 Experiments duration (total time ×10 times)

Users	Baseline	300 ms (s)	600 ms (s)	1200 ms (s)	2400 ms (s)	4800 ms (s)	15,000 ms (s)
150	60	65	70	75	80	90	180
300	110	115	120	125	135	145	235
600	210	215	225	235	245	260	350
1200	410	415	420	425	435	450	560

application. The purpose of this test is to understand how applications behave when their dependency slows down, leading to requests accumulating and causing congestion in the request queue. This type of fault can cause service delivery processes to stop working, which requires the re-issuing of service requests due to connection timeouts. In this paper, a delay latency can be set to any random amount of time in milliseconds. This delay simulates the latency experienced on slower connections using Charlesproxy. Therefore, each request is subjected to the same delay, meaning that if we assign 200 requests to hit the system in 10 s with a 300-millisecond latency delay, we expect that each request will be delayed for 300 milliseconds before reaching the server side.

4.2.2 Testing methodology and process

The approach we employed to conduct performance testing for our experiments is described in this section. We utilised JMeter to conduct, collect, and analyse performance metrics. Response time, error percentage, latency, and throughput are the four performance indicators that were obtained. In this case study, we describe how PrestaShop responds to HTTP requests. We used JMeter to send basic HTTP requests to PrestaShop and parsed HTML web pages for embedded resources like images, applets, and scripts.

Two forms of experiments were conducted: a baseline without any chaos injections and with chaos slow connection injections with various delays, 300, 600, 1200, 2400, 4800, and 15,000 ms. Each experiment was conducted ten times to ensure statistical significance, and we allocated concurrent users into four groups: 150, 300, 600, and 1200 users. We conducted a total of 280 experiments and calculated averages and standard deviations for all experiments. Standard deviations were minimal compared to averages, as we observed. We tested the microservices of the PrestaShop application hosted on AWS. We analysed and compared test results to determine the effect of chaos engineering (i.e., a slow connection delay) on cloud-native application performance.

To ensure reliable and stable data from our experiments, we conducted all experiments on the same day for each user group using the same experimental settings for both baseline and chaos engineering experiments. This was to address the variation and instability of performance testing results of microservices, following the findings of Eismann et al. [24, 36]. We overcame the cold start of microservices by exercising a warm-up strategy [37], running each test for all 280 experiments three times

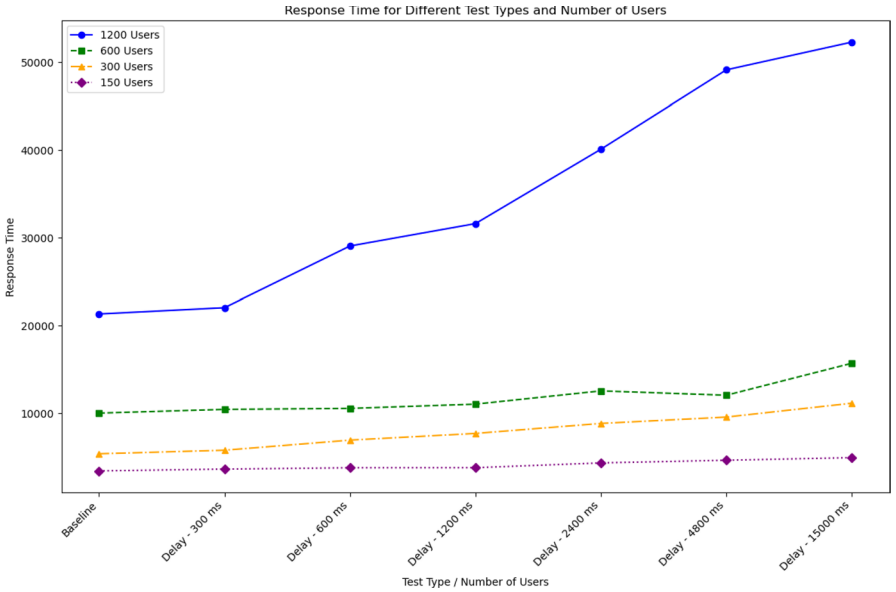


Fig. 10 Response time for PrestaShop on AWS

and then recording our results for each repeated experiment ten runs. The experiments durations for the baseline and various delays, 300, 600, 1200, 2400, 4800, and 15,000 ms are reported in Table 6.

4.2.3 Case study 2: results and discussion

This exploratory empirical study revealed the following results for the Microservices application:

Impact of delay on performance This section will present the performance measurements collected following the case study 2 experiments discussed in Sect. 4.2. The baseline data was collected from the experiments without fault injection, followed by injecting slower connections on the system under test with the delay latency times varied: 300, 600, 1200, 2400, 4800, and 15,000 ms. The average response time for each of the four user groups (150, 300, 600, and 1200) is shown in Fig. 10, which illustrates the comparison between the baseline 300, 600, 1200, 2400, 4800, and 15,000 ms delay latency experiments for the applications hosted on AWS. We note that the response time dropped between 3 and 59% compared to the baseline. The behaviour of the user groups 150, 300, and 600 were similar, with a similar performance drop. The 1200 user group performed differently, as shown in Fig. 10.

In terms of latency, Fig. 11 illustrates the values and behaviour of the application latency in all four user groups and the injected delay of 300, 600, 1200, 2400, 4800, and 15,000 ms. Compared to the baseline, we observe that the performance decreased by 30–39%. Figure 12 displays the application behaviour under test in terms of system throughput (number of requests processed in a second). We note

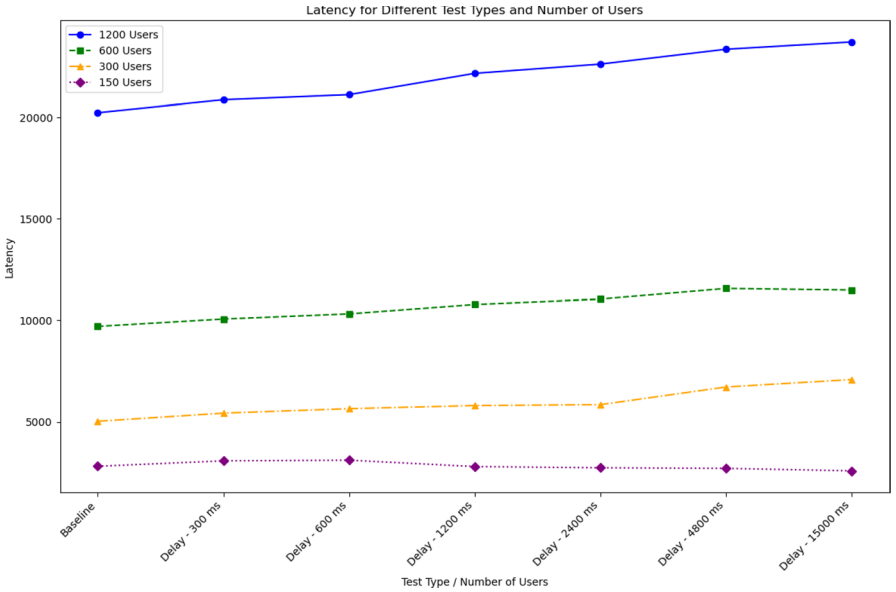


Fig. 11 Latency for PrestaShop on AWS

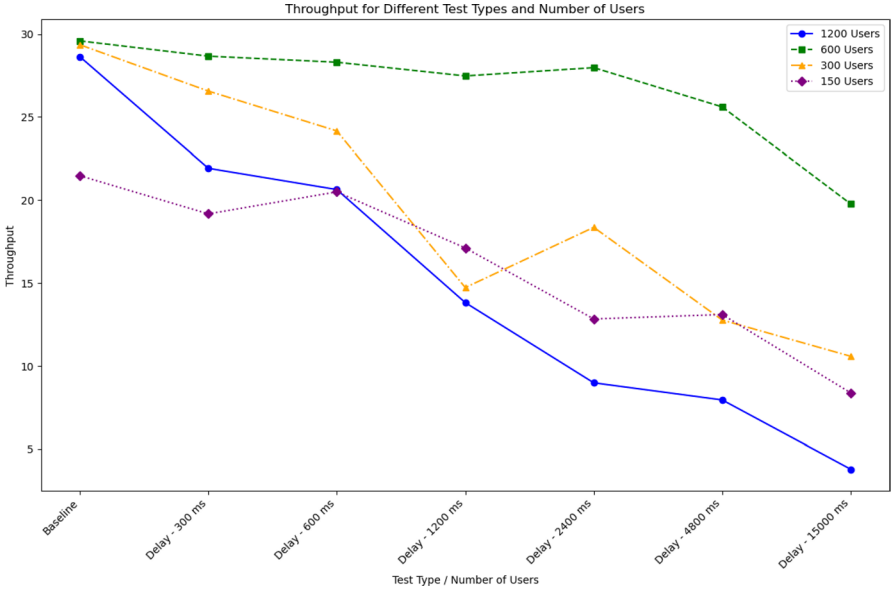


Fig. 12 Throughput for PrestaShop on AWS

different behaviours of the 300-user group as some values were less than 150 users in the experiments of the injected delay of 1200 and 4800 ms. We also note that the performance of the application in terms of throughput dropped between 3 and 87% in comparison to the baseline. We note that the 300, 600, 1200, 2400, and 4800 ms fault injection experiments have drawn similar performance behaviour in terms of response time and latency. However, we have noted a big variation in response times and latency for 1200 user groups across all fault injection delay times, especially after the 600 ms delay injections. For example, Fig. 10 for the experiments with a 600 ms delay in the 1200 user group has increased by 2.75 times compared with the 600 users, while in the same figure, which illustrates the average response times, the 600 user group compared to the 300 users the increased was by 1.5 times only. The variations between the response time and latency were higher in 1200, 2400, 4800, and 15,000 ms delays in terms of 1200 user experiments compared to the other user groups, as it reaches four times more than 600 user experiments for the same 4800 ms delay.

It is important to highlight that the faults injected across all user groups have a negative impact on throughput. In 1200, the system's throughput drops by 23% for delays of 300 ms and 87% for delays of 15,000 ms in relation to the baseline. In addition, the throughput indicator shows a significant drop in service performance because fewer requests are being processed by the system each second. Resiliency and performance suffer as a result since fewer requests are served correctly.

The performance indicator results and the calculated performance drops compared to the baseline are shown in Table 7. There is evidence that injecting a delay period into an application causes performance issues because of the resulting reduction in throughput, increase in latency, and longer response times. The performance degradation worsens when the injection time and user load increase. The results indicate that the system behaviour gradually deteriorates with stronger faults (i.e., 4800 and 15000ms delays have a greater impact than 300ms). As noted, there has been a drop in quality of between 3% and 59% for the 1200 users in terms of response time and between 23 and 87% in terms of throughput. However, in the 300-user group, the largest observed drop in latency was 7% and 29% for all fault scenarios.

The performance indicator results, together with the calculated performance drops compared to the baseline, are shown in Table 7. . There is evidence that injecting a delay period into an application causes performance issues because of the resulting reduction in throughput, increase in latency, and longer response times. The performance degradation worsens when the injection time and user load increase. Based on the measurements and performance behaviour observed during fault scenarios in this case study, it is feasible to assess the resiliency of native applications and establish a clear method for doing so.

Availability Issues We note the application began to have failures and serious availability problems, with a delay of 4800 ms and with 600 users. This was followed by more failure in the 600 user group, with a 2% when the injected delay was 15,000 ms. When the number of users increased to 1200 users, these problems increased to affect 2–4% of the users who didn't receive a service, as seen in Table 8. Eventually, the application crashes after any delay higher than 15,000 ms

Table 7 Drop-in performance compared to the baseline—case study 2

Test type	No. of users	Drop-in response time (%)	Drop-in throughput (%)	Drop in latency(%)
Delay—300 ms	1200	3	23	3
Delay—300 ms	600	4	3	4
Delay—300 ms	300	7	10	7
Delay—300 ms	150	6	11	9
Delay—600 ms	1200	27	28	4
Delay—600 ms	600	5	4	6
Delay—600 ms	300	22	18	11
Delay—600 ms	150	9	5	10
Delay—1200 ms	1200	33	52	9
Delay—1200 ms	600	9	7	10
Delay—1200 ms	300	30	50	13
Delay—1200 ms	150	10	20	1
Delay—2400 ms	1200	47	69	11
Delay—2400 ms	600	20	5	12
Delay—2400 ms	300	39	37	14
Delay—2400 ms	150	21	40	3
Delay—4800 ms	1200	57	72	13
Delay—4800 ms	600	17	13	16
Delay—4800 ms	300	44	56	25
Delay—4800 ms	150	26	39%	4
Delay—15,000 ms	1200	59	87	15
Delay—15,000 ms	600	36	33	16
Delay—15,000 ms	300	52	64	29
Delay—15,000 ms	150	31	61	− 9

Table 8 Error percentage recorded for case study 2

Lambda function/delay	No. of running users	Error percentage (%)
Baseline	1200	0.80
Delay—300 ms	1200	2
Delay—600 ms	1200	2
Delay—1200 ms	1200	2
Delay—2400 ms	1200	2
Delay—4800 ms	1200	3
Delay—15,000 ms	1200	4
Delay—4800 ms	600	1
Delay—15,000 ms	600	2

with any number of users. The unexpected issues faced by the system that impacted a significant number of users can be attributed to the combination of delays and concurrent users. When the system's performance degrades to a point where it struggles to handle the load, it may experience downtime and reduced availability. This can have adverse effects on the application's reputation, user satisfaction, and financial performance.

5 Discussion and remarks

In this paper, we demonstrate the use of chaos engineering in the form of fault injection at the production environment for two cloud-native applications hosted on AWS with different delay injections to compare runtime performance measurements for applications between the baseline (i.e. without injections) and with the injected experiments. Furthermore, we study the effect of the performance and the dependability of such native applications and how these designed faults can cause availability issues as well as the degradation of performance for the running services when such faults occur. We used two cloud-native architectures (i.e., Serverless and microservices) and two cloud-native open-source applications (Leave Management and PrestaShop) hosted on Amazon AWS. Both applications were connected with load balancing services that aim to reduce the risk of the applications experiencing performance issues, this is one of the fault tolerance mechanisms in cloud computing. We designed two chaos engineering injections, one in the form of delay latency injections within the application (case study 1), and the other with a slower connection at runtime (case study 2). Our study and results from experiments demonstrate that our exploratory methodology enables realistic evaluations of the impact of fault scenarios on the systems and explicitly quantifies the performance of the native applications under such fault scenarios with technical arguments.

In a total of 760 real-time experiments, two case studies were conducted on cloud-native serverless and containerised microservices, as well as two types of fault injections. The baseline for each case study was established without any injection, and then fault-injected experiments were conducted across four user groups: 150, 300, 600, and 1200. For the serverless application, five delay injections were demonstrated on two Lambda functions (i.e., GET and POST APIs) with 300ms, 600ms, 1200ms, 10000ms, and 27500ms delays within the function itself. On the other hand, the containerised microservices study focused on a PrestaShop application. Six slow connection delays were injected with delay times of 300ms, 600ms, 1200ms, 2400ms, 4800ms, and 15000ms. This was compared with the baseline of the same experiments for 150, 300, 600, and 1200 users.

In both case studies, we note interesting behaviour caused by the injections. The performance of both applications starts to be negatively impacted in terms of response times, latency, and throughput, starting with the smallest period of injections (300ms). As the size of the injections increased gradually, the impact worsened. Eventually, it not only affected the performance negatively but also caused dependability issues in terms of service unavailability.

As we mentioned in our availability issues discussion for both cases, we note that the serverless case study reported higher error values, up to 17% of the total requests, while the second study on microservices reported a percentage error of up to 4%. This is due to the serverless testing being more focused on the performance of the functionalities of the Lambda APIs, which is more rigorous, while the microservices were simpler and composed of basic HTTP requests to PrestaShop and parsed HTML web pages for embedded resources like images, applets, and scripts.

The findings of the first case study show that injected delays have a negative impact on performance and identify a critical threshold (27,500 ms and 300 users), after which the application begins to exhibit errors and serious availability issues. At 27,500 ms and 1200 users, the result of the unavailability of 17% (170 users of the 1000 users who are not served did not receive a service) was recorded, which indicates potential availability bottlenecks that may cause a complete application failure. The complete application failure is recorded at 28,000 ms delay with any number of users. In addition, we found that the Get Leave Request performed better than the Add Leave Request. For all user groups, the Add Leave Request was found to have issues when injecting 27,500 ms. The Get Leave Request appears to be more resilient than the Add Leave Request, creating an availability gap in the Add Leave Request.

The second case study revealed that the performance of the system suffers when delays are introduced. This impact on performance is most noticeable when a delay of 4800 ms is introduced for 600 users. Additionally, all experiments involving 1200 users reported unavailability of services between 2% to 4% of the total requests. This indicates that there are serious availability issues that can compromise the dependability of microservice applications. The application begins to record total failures after an injection larger than 15,000 ms.

In light of both case studies' results, the study's findings answer RQ1, showing that fault injection can be used to uncover dependability problems (such as availability issues) in cloud-native applications, potentially improving such applications' dependability and specifically enhancing the developed dependability models. In other words, in classical dependability analysis, dependability models are built by experts identifying potential failures and specifying the root causes for each failure using dependability analysis tools. Here, we think that developing a fault repository would be helpful. This is a set of faults established by looking at proposed failure models and identifying their underlying causes using conventional dependability analysis tools like fault tree analysis and failure mode and effect analysis. The most serious faults are listed since they may propagate to have more serious effects. Inevitably, to accomplish this, one must thoroughly understand each internal component of the application, how it functions, and any potential failures it could have. These faults are then introduced to see how they affect the system during runtime in order to pinpoint the realistic failures they cause at runtime. Before attempting to test these faults specified during traditional dependability analysis in runtime, we started by adding delays to see how they might impact the application's performance.

In such a way, the injection delay process is found to have an adverse impact on the applications' performance, which answers RQ2. However, as mentioned earlier in the results, we get unexpected outcomes that answer RQ3. More research is

still required to fully comprehend the unforeseen results of service unavailability as a result of delay injections, which were unexpected according to conventional dependability analysis. In other words, when we constructed dependability models, we didn't find any faults that would have caused a delay to be the reason the service wasn't available. Contrary to previous studies in which inserting delays merely affected performance, it was unavailable in this particular experimental trial. The worst failure modes result in a system's functionalities or the entire system becoming unavailable.

6 Conclusion and future work

This exploratory empirical study highlights the importance of employing fault injection to assess cloud-native applications' performance, resilience, and dependability, specifically the resilience and dependability of cloud-native applications, such as microservices or serverless Lambda functions. Moreover, the study answers the initial research question by demonstrating that fault injection clearly reveals dependability issues that are not detected by traditional dependability analysis.

Future studies must focus on comprehending the fundamental causes of the rapid emergence of problems at this particular delay point and investigating suitable mitigating techniques. In other words, we designed this experiment to uncover the weaknesses in the system and, specifically, the performance problems that are not uncommon under stress. However, we get unexpected results related to the observed unavailability, an important attribute of dependability. Our results reveal the weaknesses of the applications and a point of failure. It was possible to determine the thresholds at which the applications begin to encounter noticeable degradation in performance and availability concerns by doing many fault injection tests with various loads and delays. This allowed for improved planning and optimisation of native services provided by the cloud.

The unexpected absence of cloud services during chaos engineering with the lambda function's injected delay or for containerised microservices emphasises the necessity for more research into the system's performance issues and constraints. Scalability issues should also be thoroughly analysed to improve the application's resilience. This could inspire research on the scalability resilience of cloud-native applications and how scaling behaves under different fault scenarios. Further, we need to investigate and design adaptive and healing techniques for enhancing the limitations in cloud-native applications, considering the results of the injected faults and their impact on system performance. In the future, we will be working towards identifying the specific cloud-native application or platform components that are responsible for causing service unavailability. To achieve this, we will be using code instrumentation techniques. Additionally, we will develop fault-injection instrumentation that will allow us to observe how faults impact the service and cause availability issues in native applications.

Author contributions Conceptualization: ASA; Methodology: ASA and LQ; Formal analysis and investigation: ASA and LQ; Writing - original draft preparation: ASA and LQ; Writing - review and editing: [All]; Funding acquisition: ASA; Technical experiments: AZ and ASA.

Funding Researcher Development Framework award No. CSRDF22-23-1AA, Keele University, for Dr Amro Al-Said Ahmad.

Data availibility Not applicable.

Code availability Not applicable.

Declarations

Conflict of interest We declare no Conflict of interest.

Ethical approval Not applicable

Consent to participate Not applicable.

Consent for publication All Authors approved the Manuscript.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by/4.0/.

References

1. Dang LM, Piran MJ, Han D, Min K, Moon H (2019) A survey on internet of things and cloud computing for healthcare. Electronics 8(7):768. https://doi.org/10.3390/electronics8070768
2. Alabbadi MM (2011) Cloud computing for education and learning: education and learning as a service (elaas). In: 2011 14th international conference on interactive collaborative learning, pp 589–594. IEEE https://doi.org/10.1109/ICL.2011.6059655
3. Golightly L, Chang V, Xu QA, Gao X, Liu BS (2022) Adoption of cloud computing as innovation in the organization. Int J Eng Bus Manage 14:1–17. https://doi.org/10.1177/18479790221093992
4. Camacho C, Cañizares PC, Llana L, Núñez A (2022) Chaos as a software product line-a platform for improving open hybrid-cloud systems resiliency. Softw Pract Exp 52(7):1581–1614. https://doi.org/10.1002/spe.3076
5. Gokhroo MK, Govil MC, Pilli ES (2017) Detecting and mitigating faults in cloud computing environment. In: 2017 3rd international conference on computational intelligence and communication technology (CICT), pp 1–9. IEEE https://doi.org/10.1109/CIACT.2017.7977362
6. Kumari P, Kaur P (2021) A survey of fault tolerance in cloud computing. J King Saud Univ-Comput Inf Sci 33(10):1159–1176. https://doi.org/10.1016/j.jksuci.2018.09.021
7. Avizienis A, Laprie J-C, Randell B, Landwehr C (2004) Basic concepts and taxonomy of dependable and secure computing. IEEE Trans Dependable Secure Comput 1(1):11–33. https://doi.org/10.1109/TDSC.2004.2
8. Feinbube L, Pirl L, Polze A (2017) Software fault injection: a practical perspective. In: Márquez FPG, Papaelias M (eds) Dependability engineering. IntechOpen, Rijeka. Chap. https://doi.org/10.5772/intechopen.70427

9. Chowdhury A, Tripathi P (2014) Enhancing cloud computing reliability using efficient scheduling by providing reliability as a service. In: 2014 international conference on parallel, distributed and grid computing, pp 99–104. IEEE https://doi.org/10.1109/PDGC.2014.7030723
10. Sun D, Chang G, Miao C, Wang X (2013) Analyzing, modeling and evaluating dynamic adaptive fault tolerance strategies in cloud computing environments. J Supercomput 66:193–228. https://doi.org/10.1007/s11227-013-0898-7
11. Laprie J-C (2008) From dependability to resilience. In: 38th IEEE/IFIP international conference on dependable systems and networks, pp 8–9
12. Natella R, Cotroneo D, Madeira HS (2016) Assessing dependability with software fault injection: a survey. ACM Comput Surv. https://doi.org/10.1145/2841425
13. Piscitelli R, Bhasin S, Regazzoni F (2017) In: Sklavos N, Chaves R, Di Natale G, Regazzoni F (eds) Fault attacks, injection techniques and tools for simulation, pp 27–47. Springer, Cham. https://doi.org/10.1007/978-3-319-44318-8_2
14. Basiri A, Behnam N, Rooij R, Hochstein L, Kosewski L, Reynolds J, Rosenthal C (2016) Chaos engineering. IEEE Softw 33(3):35–41. https://doi.org/10.1109/MS.2016.60
15. Herscheid L, Richter D, Polze A (2015) Experimental assessment of cloud software dependability using fault injection. In: Camarinha-Matos LM, Baldissera TA, Di Orio G, Marques F (eds) Technological innovation for cloud-based engineering systems, pp 121–128. Springer, Cham. https://doi.org/10.1007/978-3-319-16766-4_13
16. Kratzke N, Quint P-C (2017) Understanding cloud-native applications after 10 years of cloud computing—a systematic mapping study. J Syst Softw 126:1–16. https://doi.org/10.1016/j.jss.2017.01.001
17. Kounev S, Reinecke P, Brosig F, Bradley JT, Joshi K, Babka V, Stefanek A, Gilmore S (2012) In: Wolter K, Avritzer A, Vieira M, Moorsel A (eds) Providing dependability and resilience in the cloud: challenges and opportunities, pp 65–81. Springer, Berlin, Heidelberg. https://doi.org/10.1007/978-3-642-29032-9_4
18. Lin S, Wang Y, Jia L (2018) System reliability assessment based on failure propagation processes. Complexity. https://doi.org/10.1155/2018/9502953
19. Herbst N, Bauer A, Kounev S, Oikonomou G, Eyk EV, Kousiouris G, Evangelinou A, Krebs R, Brecht T, Abad CL et al (2018) Quantifying cloud performance and dependability: taxonomy, metric design, and emerging challenges. ACM Trans Model Perform Eval Comput Syst 3(4):1–36. https://doi.org/10.1145/3236332
20. Armbrust M, Fox A, Griffith R, Joseph AD, Katz R, Konwinski A, Lee G, Patterson D, Rabkin A, Stoica I et al (2010) A view of cloud computing. Commun ACM 53(4):50–58. https://doi.org/10.1145/1721654.1721672
21. Zhang L, Morin B, Baudry B, Monperrus M (2021) Maximizing error injection realism for chaos engineering with system calls. IEEE Trans Dependable Secure Comput 19(4):2695–2708. https://doi.org/10.1109/TDSC.2021.3069715
22. Jaival M, Mkrtchyan K, Kaplan A (2022) Serverless cloud functions-opportunity in chaos. In: 2022 International conference on computational science and computational intelligence (CSCI), pp. 1330–1335. IEEE https://doi.org/10.1109/CSCI58124.2022.00239
23. Scheuner J, Leitner P (2020) Function-as-a-service performance evaluation: a multivocal literature review. J Syst Softw 170:110708. https://doi.org/10.1016/j.jss.2020.110708
24. Eismann S, Costa DE, Liao L, Bezemer C-P, Shang W, Hoorn A, Kounev S (2022) A case study on the stability of performance tests for serverless applications. J Syst Softw 189:111294. https://doi.org/10.1016/j.jss.2022.111294
25. Scheuner J (2022) Performance evaluation of serverless applications and infrastructures. PhD thesis, Chalmers Tekniska Hogskola (Sweden)
26. Cerveira F, Barbosa R, Madeira H, Araujo F (2020) The effects of soft errors and mitigation strategies for virtualization servers. IEEE Trans Cloud Comput 10(2):1065–1081. https://doi.org/10.1109/TCC.2020.2973146
27. Al-Said Ahmad A, Andras P (2022) Scalability resilience framework using application-level fault injection for cloud-based software services. J Cloud Comput 11(1):1–13. https://doi.org/10.1186/s13677-021-00277-z
28. Poltronieri F, Tortonesi M, Stefanelli C (2022) A chaos engineering approach for improving the resiliency of it services configurations. In: 2022 IEEE/IFIP network operations and management symposium, pp 1–6. IEEE https://doi.org/10.1109/NOMS54207.2022.9789887

29. Naqvi MA, Malik S, Astekin M, Moonen L (2022) On evaluating self-adaptive and self-healing systems using chaos engineering. In: 2022 IEEE international conference on autonomic computing and self-organizing systems (ACSOS), pp 1–10. IEEE https://doi.org/10.1109/ACSOS55765.2022.00018
30. Zhu J (2021) Serverless chaos-measuring the performance and resilience of cloud function platforms
31. Nabi M, Toeroe M, Khendek F (2016) Availability in the cloud: state of the art. J Netw Comput Appl 60:54–67. https://doi.org/10.1016/j.jnca.2015.11.014
32. Aldossary S, Allen W (2016) Data security, privacy, availability and integrity in cloud computing: issues and current solutions. Int J Adv Comput Sci Appl. https://doi.org/10.14569/IJACSA.2016.070464
33. Ding S, Wang Z, Wu D, Olson DL (2017) Utilizing customer satisfaction in ranking prediction for personalized cloud service selection. Decis Support Syst 93:1–10. https://doi.org/10.1016/j.dss.2016.09.001
34. Mukwevho MA, Celik T (2018) Toward a smart cloud: a review of fault-tolerance methods in cloud systems. IEEE Trans Serv Comput 14(2):589–605. https://doi.org/10.1109/TSC.2018.2816644
35. Waseem M, Liang P, Shahin M, Di Salle A, Márquez G (2021) Design, monitoring, and testing of microservices systems: the practitioners' perspective. J Syst Softw 182:111061. https://doi.org/10.1016/j.jss.2021.111061
36. Eismann S, Bezemer C-P, Shang W, Okanović D, Hoorn A (2020) Microservices: a performance tester's dream or nightmare? In: Proceedings of the ACM/SPEC international conference on performance engineering, pp 138–149 . https://doi.org/10.1145/3358960.3379124
37. Raj V (2023) A framework for migration of microservices based applications to serverless platform with efficient cold start latency. Authorea Preprints. https://doi.org/10.36227/techrxiv.24237196.v1

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Authors and Affiliations

Amro Al-Said Ahmad[1] · Lamis F. Al-Qora'n[2] · Ahmad Zayed[2]

✉ Amro Al-Said Ahmad
a.m.al-said.ahmad@keele.ac.uk

Lamis F. Al-Qora'n
lalqoran@philadelphia.edu.jo

Ahmad Zayed
ahmadomar19984@gmail.com

[1] School of Computer Science and Mathematics, Keele University, Staffordshire, UK

[2] Software Engineering Department, Philadelphia University, Amman, Jordan